RoadVision

Intelligent Road Monitoring System

Project Proposal

Computer-vision-based detection of visible road damage from images, recorded videos and live camera input, using the RDD2022 dataset and YOLO26 models.

Programme	B.Tech Computer Science and Engineering (AI & ML)
Project type	Academic mini project (7-day development schedule)
Team members	Ram (AI and Computer Vision) – Roll No. _____ Shakib (Application Development) – Roll No. _____
Institution	_____
Project guide	_____
Date	November 2026

Table of Contents

1. Project Overview . . . 3

2. Problem Statement . . . 3

3. Objectives . . . 3

4. Dataset . . . 4

4.1 Source and scale . . . 4

4.2 Supported classes . . . 4

4.3 Dataset preparation requirements . . . 4

5. Model Strategy . . . 5

5.1 Primary model: YOLO26m . . . 5

5.2 Fallback model: YOLO26n . . . 5

5.3 Verification before training . . . 5

5.4 Evaluation plan . . . 5

6. Input Modes . . . 6

7. Expected Output . . . 6

8. Technology Stack . . . 7

8.1 System architecture . . . 7

9. Project Scope . . . 8

10. Seven-Day Development Schedule . . . 9

11. Feasibility and Risk Management . . . 9

12. Expected Outcome . . . 10

12.1 Known limitations . . . 10

13. References and Licensing . . . 11

Licensing and attribution . . . 11

1. Project Overview

RoadVision is a computer-vision-based road monitoring application that detects visible road damage from uploaded images, recorded videos and a live webcam feed. The system draws bounding boxes around detected damage, labels each detection with its class and confidence score, and summarises the results as class-wise counts. Every inspection is stored locally so that earlier results can be reviewed.

The project is a B.Tech Computer Science and Engineering (AI & ML) academic mini project developed by a team of two students over seven days. It combines a fine-tuned YOLO26 object-detection model with a FastAPI backend and a React dashboard, and runs entirely on a local Windows 11 computer without paid cloud services.

RoadVision is intended as a preliminary, assistive screening tool for demonstration and learning. It is not an engineering-grade pavement assessment system, and its outputs should not replace inspection by qualified personnel.

2. Problem Statement

Road surfaces deteriorate continuously under traffic, weather and ageing. Identifying where damage has occurred is usually done by manual inspection, in which personnel travel along road sections, observe the surface and record defects. This approach has several practical limitations:

- **Time and effort:** inspecting long road networks on foot or by vehicle is slow and labour-intensive.
- **Inconsistency:** different inspectors may judge the same defect differently, and one inspector's judgement can vary with fatigue, lighting and viewing angle.
- **Limited traceability:** handwritten or ad-hoc records are difficult to compare across inspections.

Computer vision can assist preliminary road-damage identification. An object-detection model applied to photographs or video can locate candidate damage regions, classify them into defined categories and apply the same decision rules to every frame, producing structured and repeatable records. Such a tool does not remove the need for expert verification, but it can help prioritise which images or road sections deserve closer attention. RoadVision investigates this idea at student-project scale using a public benchmark dataset.

3. Objectives

The project has the following objectives:

- Detect four road-damage categories.
- Support image, recorded-video and live-webcam input.
- Display bounding boxes, class labels and confidence scores.
- Generate class-wise detection summaries.
- Store inspection history locally.
- Evaluate detection performance using labelled test data.
- Provide a lightweight fallback model.
- Operate without paid cloud services.

4. Dataset

4.1 Source and scale

The project uses the RDD2022 (Road Damage Detection 2022) dataset, which contains 47,420 images collected across six countries: Japan, India, the Czech Republic, Norway, the United States and China [1], [2]. The team will download and preserve the complete original dataset. Training will use a manageable, validated subset of the labelled images, chosen according to annotation availability, class distribution, hardware capacity and available time. The figure of 47,420 images describes the full dataset, not the number of images that will be used for training.

Table 1: RDD2022 sources

Item	Reference
Official dataset	https://doi.org/10.6084/m9.figshare.21431547.v1
Research paper	https://doi.org/10.1002/gdj3.260

4.2 Supported classes

Four damage classes are supported. The class codes follow the dataset's annotation scheme and will be kept identical across the primary and fallback models.

Table 2: Road-damage classes

Code	Class
D00	Longitudinal crack
D10	Transverse crack
D20	Alligator crack
D40	Pothole

4.3 Dataset preparation requirements

- Download and preserve the complete original dataset.
- Inspect dataset structure, annotations and class distribution.
- Convert Pascal VOC XML annotations into YOLO format where required.
- Separate labelled images into training, validation and testing sets.
- Split at image level and prevent data leakage.
- Keep official unlabelled test images separate; do not use them to claim quantitative evaluation.
- Select a manageable, validated subset of the labelled images for training, based on annotation availability, class distribution, hardware capacity and training time; record the selection criteria and resulting counts.
- Retain only annotations of the four supported classes (D00, D10, D20, D40); exclude and count any other labels.
- Preserve original data and annotations until the project has been successfully evaluated.

The team will not assume that every downloaded image has a usable annotation. Missing, empty, malformed or out-of-range annotation files will be identified during inspection and handled explicitly,

and the number of images actually used at each stage will be recorded in the final report.

5. Model Strategy

5.1 Primary model: YOLO26m

- Fine-tune pretrained weights on the prepared RDD2022 data (YOLO26 is provided through the Ultralytics package [3]).
- Use as the main road-damage detector.

5.2 Fallback model: YOLO26n

- Fine-tune or train for the same four road-damage classes.
- Use YOLO26n (a) at run time, if YOLO26m fails to load or does not run reliably on the target computer, and (b) as the working detector if YOLO26m cannot be fine-tuned within the available GPU memory and time. The active model is always displayed and the reason for any fallback is recorded.
- Maintain the same class mapping and output format.

A generic COCO-pretrained YOLO26n model is not a valid fallback for the four road-damage classes, because it has not been trained to recognise them. The fallback must be fine-tuned on the prepared road-damage data before use.

5.3 Verification before training

Model loading and GPU memory usage will be tested before committing to long training runs, so that batch size and image resolution are chosen from measurement rather than assumption. No accuracy or training completion is promised in this proposal; all performance statements in the final report will rest on measured results.

5.4 Evaluation plan

Performance will be evaluated on a held-out test split created from the labelled images, which is never used for training or model selection. The official RDD2022 test images are released without annotations, so results will be reported on the team's own split and are not comparable with published challenge results. Both models will be evaluated on the same split with identical settings (image size, confidence threshold and IoU threshold). Mean average precision is computed with a low confidence threshold so that the full precision-recall curve is covered, while precision and recall are reported at the confidence threshold used in the application, which will be stated. The test split is used for the final evaluation only.

Table 3: Planned evaluation measures

Measure	Purpose
Precision and recall	Reliability of detections and fraction of damage found, per class and overall
mAP at standard IoU thresholds	Overall detection quality, reported per class and averaged
Per-class results	Average precision and number of test objects for D00, D10, D20 and D40
Confusion matrix	Which damage classes are confused with each other or with background
Inference speed	Measured time per image or frame on the project hardware, for both YOLO26m and YOLO26n
GPU memory use	Measured memory during inference, supporting the fallback decision

6. Input Modes

- **Image upload and detection:** the user uploads a still image and receives an annotated result.
- **Recorded-video upload and frame-by-frame detection:** a video file is processed frame by frame and an annotated video is produced.
- **Live webcam or USB camera:** detection runs on a camera connected to the computer running the application.

7. Expected Output

Table 4: Expected outputs

Output	Description
Annotated images and videos	Original media with detections drawn on it
Bounding boxes	Boxes around each detected damage region
Class labels	One of the four damage classes for each box
Confidence scores	Model confidence for each detection
Class-wise counts	Number of detections per damage class for each inspection
Inspection history	Past inspections stored locally and viewable in the dashboard
Evaluation metrics	Computed on held-out labelled data and reported in the final report

8. Technology Stack

Table 5: Technology stack

Component	Technology
Object detection	YOLO26m and YOLO26n
Deep learning	PyTorch
Image/video processing	OpenCV
Backend	Python, FastAPI, Uvicorn
Frontend	React, TypeScript, Vite
Styling	Tailwind CSS
Database	SQLite
GPU acceleration	Compatible NVIDIA CUDA-enabled PyTorch
Operating environment	Windows 11
Detection framework	Ultralytics Python package (YOLO26 implementation)
Development hardware	ASUS ROG Zephyrus G14; AMD Ryzen 9; NVIDIA RTX 5060 Laptop GPU; 16 GB RAM

ONNX Runtime and ByteTrack are optional enhancements, not core requirements. The deep-learning stack uses PyTorch [4] with the Ultralytics YOLO26 implementation [3].

8.1 System architecture

Figure 1 shows the planned architecture. The React frontend sends user input to the FastAPI backend, which passes it to the detection engine. The engine uses OpenCV for frame handling and annotation and PyTorch to run the YOLO26 model, and results are returned to the dashboard and saved locally. Model weights are produced separately by the offline preparation pipeline.

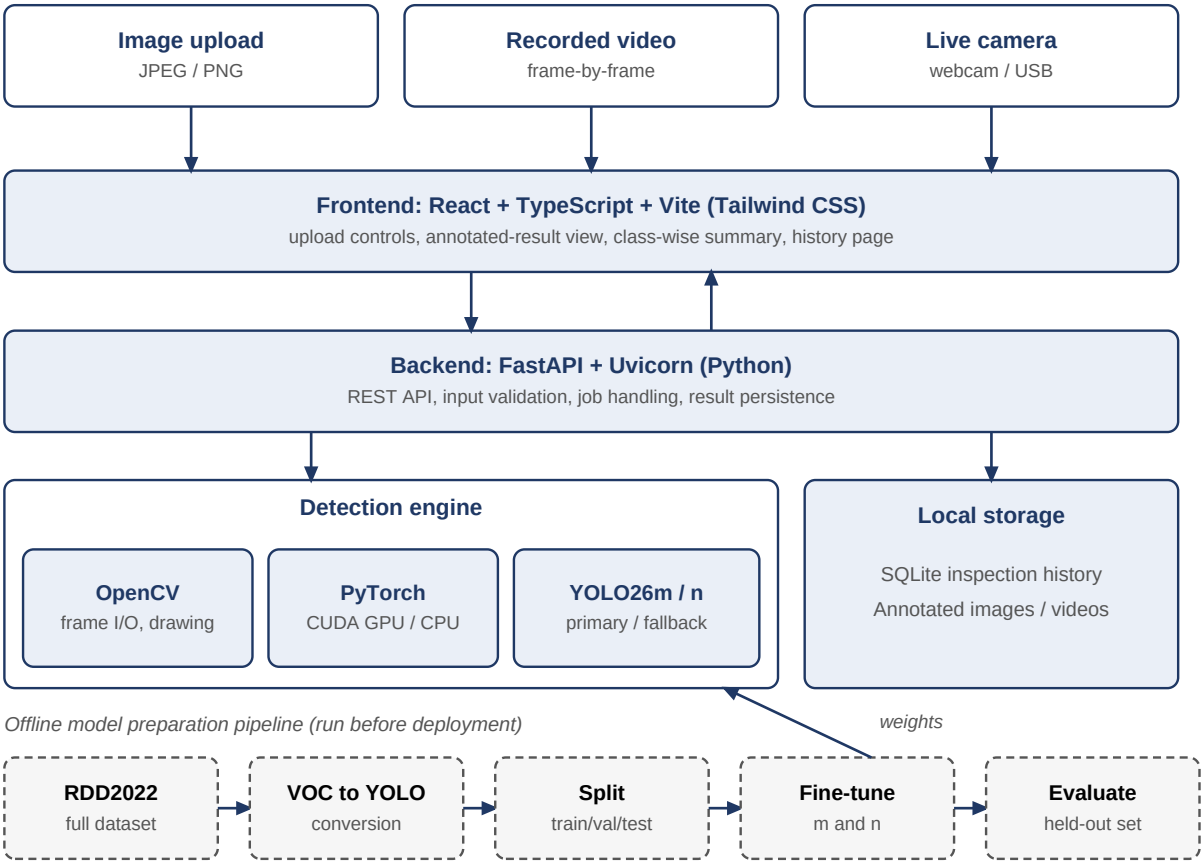

Figure 1: RoadVision system architecture

9. Project Scope

Table 6: Scope of the project

Included	Excluded
Complete RDD2022 dataset acquisition	GPS tracking and interactive maps
Annotation inspection and conversion	Mobile application
Dataset preparation and splitting	Remote CCTV integration
YOLO26m primary model and YOLO26n fallback experiment	Pothole-depth measurement
Image, video and webcam detection	Predictive maintenance
Annotated results and class-wise summaries	Municipal complaint integration
Local SQLite inspection history	Local road-data collection
React dashboard and FastAPI backend	

10. Seven-Day Development Schedule

Table 7: Day-wise plan

Day	Ram – AI and Computer Vision	Shakib – Application Development
1	Environment setup, dataset download/inspection and model smoke test	Frontend/backend setup and UI skeleton
2	Annotation conversion, dataset preparation and splitting	Image upload interface and API contract
3	YOLO26m pilot fine-tuning on the prepared subset (longer runs continue in the background) and preliminary validation	FastAPI integration and image results, built against the shared inference interface using the Day 1 smoke-test model (not presented as a road-damage result)
4	Video inference and annotated output	Video upload, results view and SQLite history
5	YOLO26n fallback experiment and live-camera inference	Live-camera interface and integration
6	Final evaluation on the held-out test split, performance tuning and bug fixes	Dashboard polish and integration testing
7	Final validation and technical explanation	Demo rehearsal, README and report preparation

Training runs start as soon as the prepared data allows and continue in the background while application development proceeds; the daily plan lists checkpoints, not guaranteed completion dates. A decision point is set at the end of Day 3: if YOLO26m cannot be fine-tuned at a usable batch size and image resolution on the available GPU, YOLO26n becomes the working detector and this change is documented.

Both members participate in integration, testing and the final demonstration. Responsibilities are provisional and may be adjusted based on each member's skills.

11. Feasibility and Risk Management

The complete dataset will be downloaded and preserved, but it will not all be used in training. Experiments will be staged: a small pilot subset first, to verify the pipeline and measure GPU memory and speed, then a larger validated subset sized to the available time. The final report will state exactly how many images and objects were used for training, validation and testing.

The development laptop has an NVIDIA RTX 5060 Laptop GPU with 8 GB of video memory, which is the limiting resource for training YOLO26m. Before full training, GPU memory use will be measured for different batch sizes and image resolutions in a short pilot run, and settings will be chosen from those measurements. Training and application testing share this single GPU, so long training runs will be scheduled in the background and live-camera testing will be done when training is not running.

Table 8: Feasibility considerations and mitigation

Consideration	Risk	Planned response
Dataset storage	The complete dataset, plus converted copies, needs substantial disk space.	Check free disk space before download; keep original files untouched and store converted data separately.
GPU memory and training time	YOLO26m may not fit in GPU memory or may train too slowly.	Test loading and memory first; adjust batch size and image resolution; fall back to YOLO26n if needed.
Annotation conversion and validation	Some images may lack usable annotations; conversion errors could corrupt labels.	Validate every converted file; exclude unusable items and record the counts.
Class imbalance	Some damage classes may be much rarer than others, biasing the model.	Inspect distribution early; report per-class metrics, not only averages.
Batch size and resolution	Larger settings improve detail but raise memory and time cost.	Select from short trial runs on the project hardware.
GPU software compatibility	RTX 50-series (Blackwell) GPUs need a PyTorch build with CUDA 12.8 or newer; older builds fail with an unsupported-GPU error.	On Day 1, install PyTorch from the official selector, confirm that torch.cuda.is_available() returns True, and run a short GPU test before relying on the GPU.
Two model variants in seven days	Training and evaluating both models may not fit in the schedule.	Use staged experiments; prioritise a working primary model.

If time is limited, the team will prioritise a working primary model and document any incomplete fallback experiment honestly, rather than presenting unfinished work as complete.

12. Expected Outcome

The expected outcome is a locally running web application that identifies four selected road-damage categories from images, recorded videos and live webcam input.

The final report will present actual measured evaluation metrics, runtime measurements, screenshots and limitations. Results will not be fabricated, and the system will not be presented as an engineering-grade road assessment.

12.1 Known limitations

- Results apply to RDD2022-style imagery and may not transfer to local roads with different cameras, lighting or pavement types.
- Only four damage classes are detected; other damage types are not recognised.
- The system does not estimate severity, size or depth.
- Training uses a subset of the dataset, and class imbalance may reduce performance on rarer classes.
- Confidence scores are model outputs, not guaranteed probabilities of correctness.
- Live-camera speed depends on the computer used.
- The system is a preliminary visual screening aid and is not an engineering-grade road assessment.

13. References and Licensing

[1] RDD2022 dataset (Figshare): https://doi.org/10.6084/m9.figshare.21431547.v1

[2] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal and Y. Sekimoto, "RDD2022: A multi-national image dataset for automatic road damage detection," Geoscience Data Journal, vol. 11, no. 4, pp. 846–862, 2024. https://doi.org/10.1002/gdj3.260

[3] YOLO26 documentation: https://docs.ultralytics.com/models/yolo26

[4] PyTorch: https://pytorch.org/get-started/locally/

[5] Ultralytics licensing: https://www.ultralytics.com/license

Licensing and attribution

RDD2022 is used under the licence terms stated on its Figshare record, which will be quoted in the final report; its authors are credited by citing [1] and [2] in the report and README. YOLO26 is provided by Ultralytics under the GNU AGPL-3.0 licence, with a separate Enterprise licence for commercial use [5], and its pretrained weights were trained on the COCO dataset. This academic project uses it under AGPL-3.0; the team will follow its conditions, including those on distribution and network use, and will not describe the project as commercially licensed. PyTorch, OpenCV, FastAPI, React, SQLite and other dependencies will be listed with their licences in the README. These terms will be re-checked before the project is distributed.